Lecture 7

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

August 5, 2026

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?
- Solution to this problem is the basis of the lexical analyzers.

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?
- Solution to this problem is the basis of the lexical analyzers.
- Just the yes/no answer is not important.

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?
- Solution to this problem is the basis of the lexical analyzers.
- Just the yes/no answer is not important.
- **Goal: Partition the input into tokens**.

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?
- Solution to this problem is the basis of the lexical analyzers.
- Just the yes/no answer is not important.
- **Goal: Partition the input into tokens**.

Algorithm

1. Construct R merging all the regular expressions
   $R = R_1 + R_2 + R_3 + \ldots$

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?
- Solution to this problem is the basis of the lexical analyzers.
- Just the yes/no answer is not important.
- **Goal: Partition the input into tokens**.

Algorithm

1. Construct R merging all the regular expressions
   $R = R_1 + R_2 + R_3 + \ldots$
2. Let input be $x_1 \ldots x_n$ then task is for $1 \leq i \leq n$ check $x_1 \ldots x_i \in L(R)$

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?
- Solution to this problem is the basis of the lexical analyzers.
- Just the yes/no answer is not important.
- **Goal: Partition the input into tokens**.

Algorithm

1. Construct R merging all the regular expressions
   $R = R_1 + R_2 + R_3 + \ldots$
2. Let input be $x_1 \ldots x_n$ then task is for $1 \leq i \leq n$ check $x_1 \ldots x_i \in L(R)$
3. $x_1 \ldots x_i \in L(R) \rightarrow x_1 \ldots x_i \in L(R_j)$ for some j. smallest such j is token class of $x_1 \ldots x_i$

Regular expressions in specifications

- Regular expressions are only specifications; implementation is still required.
- Given a string s and a regular expression R, does $s \in L(R)$?
- Solution to this problem is the basis of the lexical analyzers.
- Just the yes/no answer is not important.
- **Goal: Partition the input into tokens**.

Algorithm

1. Construct R merging all the regular expressions $R = R_1 + R_2 + R_3 + \ldots$
2. Let input be $x_1 \ldots x_n$ then task is for $1 \leq i \leq n$ check $x_1 \ldots x_i \in L(R)$
3. $x_1 \ldots x_i \in L(R) \rightarrow x_1 \ldots x_i \in L(R_j)$ for some j. smallest such j is token class of $x_1 \ldots x_i$
4. Remove $x_1 \ldots x_i$ from input; go to (2)

Cont...

- The algorithm gives priority to tokens listed earlier

Cont...

- The algorithm gives priority to tokens listed earlier
  Treats `if` as a keyword and not an identifier

Cont...

- The algorithm gives priority to tokens listed earlier
  Treats `if` as a keyword and not an identifier
- How much input is used? What if
  - $x_1 \dots x_i \in L(R)$
  - $x_1 \dots x_j \in L(R)$

Cont...

- The algorithm gives priority to tokens listed earlier
  Treats `if` as a keyword and not an identifier
- How much input is used? What if
  - $x_1 \dots x_i \in L(R)$
  - $x_1 \dots x_j \in L(R)$
  - Pick up the longest possible string in $L(R)$

Cont...

- The algorithm gives priority to tokens listed earlier
  Treats `if` as a keyword and not an identifier
- How much input is used? What if
  - $x_1 \dots x_i \in L(R)$
  - $x_1 \dots x_j \in L(R)$
  - Pick up the longest possible string in $L(R)$
  - The principle of **maximal munch**
- Regular expressions provide a concise and useful notation for string patterns

Cont...

- The algorithm gives priority to tokens listed earlier
  Treats `if` as a keyword and not an identifier
- How much input is used? What if
  - $x_1 \dots x_i \in L(R)$
  - $x_1 \dots x_j \in L(R)$
  - Pick up the longest possible string in $L(R)$
  - The principle of **maximal munch**
- Regular expressions provide a concise and useful notation for string patterns
- Good algorithms require a single pass over the input

Extended Regular Expression

Extended Regular Expression

- One or more instances

Extended Regular Expression

- One or more instances

  $r^+ = rr*$

Extended Regular Expression

- One or more instances
  $$r^+ = rr*$$
- Zero or one instance

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character
  .

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character
  .
- Beginning of the line

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character
  .
- Beginning of the line
  ^

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character
  .
- Beginning of the line
  ^
- End of the line

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character
  .
- Beginning of the line
  ^
- End of the line
  $

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character
  .
- Beginning of the line
  ^
- End of the line
  $
- Between m and n occurrence

Extended Regular Expression

- One or more instances
  $r^+ = rr*$
- Zero or one instance
  r?
- Character classes
  $[a_1 a_2 \ldots a_n]$
- Any character
  .
- Beginning of the line
  ^
- End of the line
  $
- Between m and n occurrence
  r{m,n}

Lexical analyzer generator

- Input to the generator

Lexical analyzer generator

- Input to the generator
  - List of regular expressions in priority order

Lexical analyzer generator

- Input to the generator
  - List of regular expressions in priority order
  - Associated actions for each of regular expression (generates kind of token and other bookkeeping information)

Lexical analyzer generator

- Input to the generator
  - List of regular expressions in priority order
  - Associated actions for each of regular expression (generates kind of token and other bookkeeping information)
- Output of the generator

Lexical analyzer generator

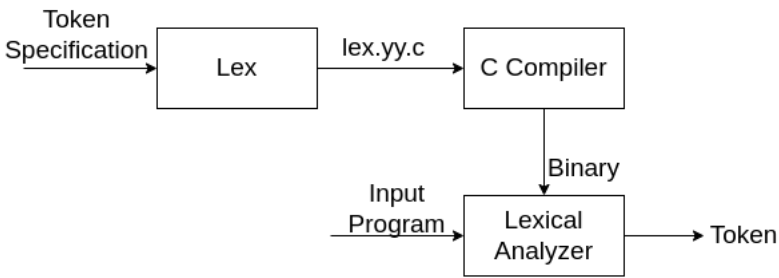
- Input to the generator
  - List of regular expressions in priority order
  - Associated actions for each of regular expression (generates kind of token and other bookkeeping information)
- Output of the generator
  - Program that reads input character stream and breaks that into tokens

Lexical analyzer generator

- Input to the generator
  - List of regular expressions in priority order
  - Associated actions for each of regular expression (generates kind of token and other bookkeeping information)
- Output of the generator
  - Program that reads input character stream and breaks that into tokens
  - Reports lexical errors (unexpected characters), if any

Lexical analyzer generator

- Input to the generator
  - List of regular expressions in priority order
  - Associated actions for each of regular expression (generates kind of token and other bookkeeping information)
- Output of the generator
  - Program that reads input character stream and breaks that into tokens
  - Reports lexical errors (unexpected characters), if any



Sample Lex File

File Format

declaration
%%
transition rules
%%
auxiliary functions

Sample Lex File

File Format

declaration
%%
transition rules
%%
auxiliary functions

Declaration

- Variables which is going to be used in the rules must be defined in this section.

Sample Lex File

File Format

declaration
%%
transition rules
%%
auxiliary functions

Declaration

- Variables which is going to be used in the rules must be defined in this section.
  D [0-9]

Sample Lex File

File Format

declaration
%%
transition rules
%%
auxiliary functions

Declaration

- Variables which is going to be used in the rules must be defined in this section.
  D [0-9]
- Section enclosed in %{%} delimiter lines are copied to the lex.yy.c file.

Sample Lex File

File Format

declaration
%%
transition rules
%%
auxiliary functions

Declaration

- Variables which is going to be used in the rules must be defined in this section.
  D [0-9]
- Section enclosed in %{%} delimiter lines are copied to the lex.yy.c file.
  %{
  include < *math.h* >
  int count;
  %}

Sample Lex file

Transition Rule

Pattern $\{Action\}$

Sample Lex file

Transition Rule

Pattern $\{Action\}$

Auxiliary Functions

C Code, going directly into `lex.yy.c`

Sample Lex file

Transition Rule

Pattern $\{Action\}$

Auxiliary Functions

C Code, going directly into `lex.yy.c`

Important functions/variables

- `yyin`

Sample Lex file

Transition Rule

Pattern {*Action*}

Auxiliary Functions

C Code, going directly into `lex.yy.c`

Important functions/variables

- `yyin`
- `yylex`

Sample Lex file

Transition Rule

Pattern {*Action*}

Auxiliary Functions

C Code, going directly into `lex.yy.c`

Important functions/variables

- `yyin`
- `yylex`
- `yytext`

Sample Lex file

Transition Rule

Pattern {*Action*}

Auxiliary Functions

C Code, going directly into `lex.yy.c`

Important functions/variables

- `yyin`
- `yylex`
- `yytext`
- `yyerror`

Sample Lex file

Transition Rule

Pattern {*Action*}

Auxiliary Functions

C Code, going directly into `lex.yy.c`

Important functions/variables

- `yyin`
- `yylex`
- `yytext`
- `yyerror`
- `yyleng`

Sample Lex file

Transition Rule

Pattern $\{Action\}$

Auxiliary Functions

C Code, going directly into `lex.yy.c`

Important functions/variables

- `yyin`
- `yylex`
- `yytext`
- `yyerror`
- `yyleng`
- `yylineno`

How lex works?

- Regular expressions describe the languages that can be recognized by finite automata

How lex works?

- Regular expressions describe the languages that can be recognized by finite automata
- Translate each token regular expression into a non-deterministic finite automaton

How lex works?

- Regular expressions describe the languages that can be recognized by finite automata
- Translate each token regular expression into a non-deterministic finite automaton
- Convert the NFA into an equivalent DFA

How lex works?

- Regular expressions describe the languages that can be recognized by finite automata
- Translate each token regular expression into a non-deterministic finite automaton
- Convert the NFA into an equivalent DFA
- Minimize the DFA to reduce number of states

How lex works?

- Regular expressions describe the languages that can be recognized by finite automata
- Translate each token regular expression into a non-deterministic finite automaton
- Convert the NFA into an equivalent DFA
- Minimize the DFA to reduce number of states
- Emit code driven by the DFA tables

How to break up text

- How to tokenize $elsex = 0$

How to break up text

- How to tokenize $elsex = 0$
  - $else\ x = 0$
  - $elsex = 0$
- Regular expressions alone are not enough

How to break up text

- How to tokenize $elsex = 0$
  - $else\ x = 0$
  - $elsex = 0$
- Regular expressions alone are not enough
- the longest match wins

How to break up text

- How to tokenize $elsex = 0$
  - $else\ x = 0$
  - $elsex = 0$
- Regular expressions alone are not enough
- the longest match wins
- Ties are resolved by prioritizing tokens

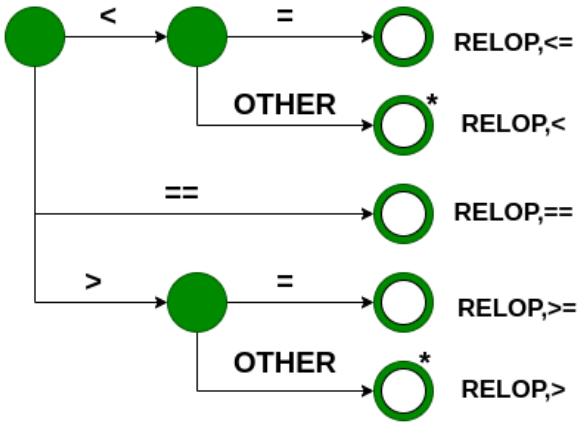
How to break up text

- How to tokenize *elsex* = 0
  - *else* $x = 0$
  - *elsex* $= 0$
- Regular expressions alone are not enough
- the longest match wins
- Ties are resolved by prioritizing tokens
- Lexical definitions consist of **regular definitions, priority rules**, and **maximal munch principle**

How to break up text

- How to tokenize *elsex* = 0
  - *else* $x = 0$
  - *elsex* $= 0$
- Regular expressions alone are not enough
- the longest match wins
- Ties are resolved by prioritizing tokens
- Lexical definitions consist of **regular definitions, priority rules**, and **maximal munch principle**
- Construct an analyzer that will return ⟨`token, lexeme`⟩ pairs

Transition Diagram for Relational Operator



Correctness check

- The lexeme for a given token must be the longest possible

Correctness check

- The lexeme for a given token must be the longest possible
- Assume input to be 12.34E56

Correctness check

- The lexeme for a given token must be the longest possible
- Assume input to be 12.34E56
- It can be matched as $\langle$`int`, 12$\rangle$

Correctness check

- The lexeme for a given token must be the longest possible
- Assume input to be 12.34E56
- It can be matched as $\langle$`int`, 12$\rangle$
- the matching should always start with the first transition diagram

Correctness check

- The lexeme for a given token must be the longest possible
- Assume input to be 12.34E56
- It can be matched as $\langle$`int`, 12$\rangle$
- the matching should always start with the first transition diagram
- If failure occurs in one transition diagram, then retract the forward pointer to the start state and activate the next diagram

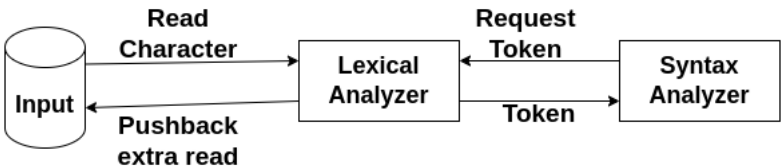
Correctness check

- The lexeme for a given token must be the longest possible
- Assume input to be 12.34E56
- It can be matched as $\langle$`int`, 12$\rangle$
- the matching should always start with the first transition diagram
- If failure occurs in one transition diagram, then retract the forward pointer to the start state and activate the next diagram
- If failure occurs in all diagrams, then a lexical error has occurred

Correctness check

- The lexeme for a given token must be the longest possible
- Assume input to be 12.34E56
- It can be matched as ⟨`int`, 12⟩
- the matching should always start with the first transition diagram
- If failure occurs in one transition diagram, then retract the forward pointer to the start state and activate the next diagram
- If failure occurs in all diagrams, then a lexical error has occurred
- This can be implemented using **lots of** `switch-cases` in C programming language.

Interface to other passes



Syntax Analysis

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

August 5, 2026

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers
- To check whether variables are of types on which operations are allowed

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers
- To check whether variables are of types on which operations are allowed ✗

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers
- To check whether variables are of types on which operations are allowed ✗
- To check whether a variable has been declared before use

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers
- To check whether variables are of types on which operations are allowed ✗
- To check whether a variable has been declared before use ✗

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers
- To check whether variables are of types on which operations are allowed ✗
- To check whether a variable has been declared before use ✗
- To check whether a variable has been initialized

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers

- To check whether variables are of types on which operations are allowed ✗
- To check whether a variable has been declared before use ✗
- To check whether a variable has been initialized ✗

Overview of Syntax Analysis

- Check syntax and construct abstract syntax tree
- Error reporting and recovery
- Model using context-free grammars
- Recognize using push-down automata/table-driven Parsers

- To check whether variables are of types on which operations are allowed ✗
- To check whether a variable has been declared before use ✗
- To check whether a variable has been initialized ✗
- These issues will be handled in semantic analysis

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$$
\begin{aligned}
\textit{list} &\rightarrow \textit{list} + \textit{digit} \\
&\quad | \textit{list} - \textit{digit} \\
&\quad | \textit{digit} \\
\textit{digit} &\rightarrow 0|1|2 \ldots |9
\end{aligned}
$$

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$list \rightarrow list + digit$
$\quad\quad | list - digit$
$\quad\quad | digit$
$digit \rightarrow 0|1|2 \ldots |9$

String Derivation:

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$list \rightarrow list + digit$
$\quad\quad | list - digit$
$\quad\quad | digit$
$digit \rightarrow 0|1|2 \ldots |9$

String Derivation:

$list \rightarrow \underline{list} + digit$

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$list \rightarrow list + digit$
$\quad\quad | list - digit$
$\quad\quad | digit$
$digit \rightarrow 0|1|2 \ldots |9$

String Derivation:

$list \rightarrow \underline{list} + digit$
$list \rightarrow \underline{list} - digit + digit$

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$list \rightarrow list + digit$
$\quad\quad | list - digit$
$\quad\quad | digit$
$digit \rightarrow 0|1|2 \ldots |9$

String Derivation:

$list \rightarrow \underline{list} + digit$
$list \rightarrow \underline{list} - digit + digit$
$list \rightarrow \underline{digit} - digit + digit$

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$$
\begin{aligned}
list &\rightarrow list + digit \\
&\quad | list - digit \\
&\quad | digit \\
digit &\rightarrow 0|1|2 \ldots |9
\end{aligned}
$$

String Derivation:

$$
\begin{aligned}
list &\rightarrow \underline{list} + digit \\
list &\rightarrow \underline{list} - digit + digit \\
list &\rightarrow \underline{digit} - digit + digit \\
list &\rightarrow 9 - \underline{digit} + digit
\end{aligned}
$$

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$list \rightarrow list + digit$
$\quad\quad | list - digit$
$\quad\quad | digit$
$digit \rightarrow 0|1|2 \ldots |9$

String Derivation:

$list \rightarrow \underline{list} + digit$
$list \rightarrow \underline{list} - digit + digit$
$list \rightarrow \underline{digit} - digit + digit$
$list \rightarrow 9 - \underline{digit} + digit$
$list \rightarrow 9 - 5 + \underline{digit}$

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$$
\begin{aligned}
list &\rightarrow list + digit \\
&\quad | list - digit \\
&\quad | digit \\
digit &\rightarrow 0|1|2 \ldots |9
\end{aligned}
$$

String Derivation:

$$
\begin{aligned}
list &\rightarrow \underline{list} + digit \\
list &\rightarrow \underline{list} - digit + digit \\
list &\rightarrow \underline{digit} - digit + digit \\
list &\rightarrow 9 - \underline{digit} + digit \\
list &\rightarrow 9 - 5 + \underline{digit} \\
list &\rightarrow 9 - 5 + 2
\end{aligned}
$$

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$list \rightarrow list + digit$
$\quad\quad | list - digit$
$\quad\quad | digit$
$digit \rightarrow 0|1|2 \ldots |9$

String Derivation:

$list \rightarrow \underline{list} + digit$
$list \rightarrow \underline{list} - digit + digit$
$list \rightarrow \underline{digit} - digit + digit$
$list \rightarrow 9 - \underline{digit} + digit$
$list \rightarrow 9 - 5 + \underline{digit}$
$list \rightarrow 9 - 5 + 2$

- Which non-terminal should I choose?

Derivation

Does $9 - 5 + 2$ belong to the following grammar?

$list \rightarrow list + digit$
$\quad\quad | list - digit$
$\quad\quad | digit$
$digit \rightarrow 0|1|2 \ldots |9$

String Derivation:

$list \rightarrow \underline{list} + digit$
$list \rightarrow \underline{list} - digit + digit$
$list \rightarrow \underline{digit} - digit + digit$
$list \rightarrow 9 - \underline{digit} + digit$
$list \rightarrow 9 - 5 + \underline{digit}$
$list \rightarrow 9 - 5 + 2$

- Which non-terminal should I choose?
- Which production rule should I select?

Derivation

- If there is a production $A \rightarrow \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$

Derivation

- If there is a production $A \rightarrow \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$
- $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$ is a production

Derivation

- If there is a production $A \to \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$
- $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \to \gamma$ is a production
- If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ then $\alpha_1 \Rightarrow^+ \alpha_n$

Derivation

- If there is a production $A \rightarrow \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$
- $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$ is a production
- If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ then $\alpha_1 \Rightarrow^+ \alpha_n$
- If $S \Rightarrow^+ \alpha$ where α is a string of terminals and non-terminals of G then we say that α is a **sentential** form of G.

Derivation

- If there is a production $A \rightarrow \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$
- $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$ is a production
- If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ then $\alpha_1 \Rightarrow^+ \alpha_n$
- If $S \Rightarrow^+ \alpha$ where α is a string of terminals and non-terminals of G then we say that α is a **sentential** form of G.
- If in a sentential form only the leftmost non terminal is replaced then it becomes leftmost derivation

Derivation

- If there is a production $A \rightarrow \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$
- $\alpha A \beta \Rightarrow \alpha\gamma\beta$ if $A \rightarrow \gamma$ is a production
- If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ then $\alpha_1 \Rightarrow^+ \alpha_n$
- If $S \Rightarrow^+ \alpha$ where α is a string of terminals and non-terminals of G then we say that α is a **sentential** form of G.
- If in a sentential form only the leftmost non terminal is replaced then it becomes leftmost derivation
- Every leftmost step can be written as $wA\gamma \Rightarrow^{lm*} w\delta\gamma$ where w is a string of terminals and $A \rightarrow \delta$ is a production.

Derivation

- If there is a production $A \rightarrow \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$
- $\alpha A \beta \Rightarrow \alpha\gamma\beta$ if $A \rightarrow \gamma$ is a production
- If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ then $\alpha_1 \Rightarrow^+ \alpha_n$
- If $S \Rightarrow^+ \alpha$ where α is a string of terminals and non-terminals of G then we say that α is a **sentential** form of G.
- If in a sentential form only the leftmost non terminal is replaced then it becomes leftmost derivation
- Every leftmost step can be written as $wA\gamma \Rightarrow^{lm*} w\delta\gamma$ where w is a string of terminals and $A \rightarrow \delta$ is a production.
- Similarly, right most derivation can be defined.

Derivation

- If there is a production $A \rightarrow \alpha$ then we say that A derives α and is denoted by $A \Rightarrow \alpha$
- $\alpha A \beta \Rightarrow \alpha\gamma\beta$ if $A \rightarrow \gamma$ is a production
- If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ then $\alpha_1 \Rightarrow^+ \alpha_n$
- If $S \Rightarrow^+ \alpha$ where α is a string of terminals and non-terminals of G then we say that α is a **sentential** form of G.
- If in a sentential form only the leftmost non terminal is replaced then it becomes leftmost derivation
- Every leftmost step can be written as $wA\gamma \Rightarrow^{lm*} w\delta\gamma$ where w is a string of terminals and $A \rightarrow \delta$ is a production.
- Similarly, right most derivation can be defined.
- An ambiguous grammar is one that produces more than one leftmost/rightmost derivation of a sentence

Parse Tree

- It shows how the start symbol of a grammar derives a string in the language.

Parse Tree

- It shows how the start symbol of a grammar derives a string in the language.
- *root* is labeled by the start symbol

Parse Tree

- It shows how the start symbol of a grammar derives a string in the language.
- *root* is labeled by the start symbol
- *leaf* nodes are labeled by tokens

Parse Tree

- It shows how the start symbol of a grammar derives a string in the language.
- *root* is labeled by the start symbol
- *leaf* nodes are labeled by tokens
- Each internal node is labeled by a non-terminal

Parse Tree

- It shows how the start symbol of a grammar derives a string in the language.
- *root* is labeled by the start symbol
- *leaf* nodes are labeled by tokens
- Each internal node is labeled by a non-terminal
- If A is a non-terminal labeling an internal node and $x_1, x_2, \ldots x_n$ are labels of the children of that node, then $A \rightarrow x_1 x_2 \ldots x_n$ is a production

Ambiguity

Ambiguity

- A Grammar can have more than one parse tree for a string

Ambiguity

- A Grammar can have more than one parse tree for a string
- Consider grammar

Ambiguity

- A Grammar can have more than one parse tree for a string
- Consider grammar

$$\begin{aligned} string &\rightarrow string + string \\ &\mid string - string \\ &\mid 0 \mid 1 \mid \cdots \mid 9 \end{aligned}$$

Ambiguity

- A Grammar can have more than one parse tree for a string
- Consider grammar

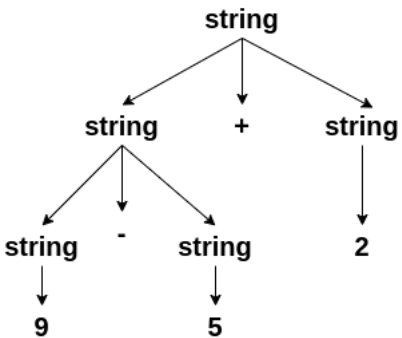
$$\begin{aligned} string &\rightarrow string + string \\ &| string - string \\ &| 0 | 1 | \cdots | 9 \end{aligned}$$

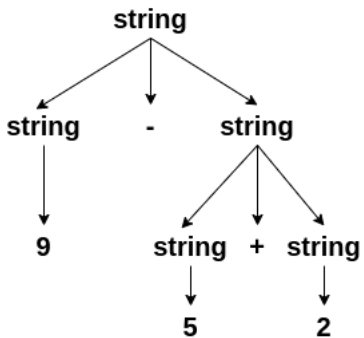
- String $9 - 5 + 2$ has two parse trees

Ambiguity

- A Grammar can have more than one parse tree for a string
- Consider grammar

  $string \rightarrow string + string$
  $\quad | string - string$
  $\quad | 0 | 1 | \cdots | 9$
- String $9 - 5 + 2$ has two parse trees





Ambiguity

- Ambiguity is problematic because meaning of the programs can be incorrect

Ambiguity

- Ambiguity is problematic because meaning of the programs can be incorrect
- Ambiguity can be handled in several ways

Ambiguity

- Ambiguity is problematic because meaning of the programs can be incorrect
- Ambiguity can be handled in several ways
  - Enforce associativity and precedence

Ambiguity

- Ambiguity is problematic because meaning of the programs can be incorrect
- Ambiguity can be handled in several ways
  - Enforce associativity and precedence
  - Rewrite the grammar (cleanest way)

Ambiguity

- Ambiguity is problematic because meaning of the programs can be incorrect
- Ambiguity can be handled in several ways
  - Enforce associativity and precedence
  - Rewrite the grammar (cleanest way)
- There are no general techniques for handling ambiguity

Ambiguity

- Ambiguity is problematic because meaning of the programs can be incorrect
- Ambiguity can be handled in several ways
  - Enforce associativity and precedence
  - Rewrite the grammar (cleanest way)
- There are no general techniques for handling ambiguity
- It is impossible to convert automatically an ambiguous grammar to an unambiguous one.

Associativity and Precedence

- If an operand has operators on both of the sides, the side on which operators takes this operand is the associativity of that operator.

Associativity and Precedence

- If an operand has operators on both of the sides, the side on which operators takes this operand is the associativity of that operator.
- In $a + b + c$ b is taken by left $+$

Associativity and Precedence

- If an operand has operators on both of the sides, the side on which operators takes this operand is the associativity of that operator.
- In $a + b + c$ b is taken by left $+$
- $+, -, *, /$ are left associative

Associativity and Precedence

- If an operand has operators on both of the sides, the side on which operators takes this operand is the associativity of that operator.
- In $a + b + c$ b is taken by left $+$
- $+, -, *, /$ are left associative
- ^= are right associative

Associativity and Precedence

- If an operand has operators on both of the sides, the side on which operators takes this operand is the associativity of that operator.
- In $a + b + c$ b is taken by left $+$
- $+, -, *, /$ are left associative
- ^= are right associative
- String a+5*2 has two possible interpretations because of two different parse trees corresponding to $(a + 5) * 2$ and $a + (5 * 2)$.

Associativity and Precedence

- If an operand has operators on both of the sides, the side on which operators takes this operand is the associativity of that operator.
- In $a + b + c$ b is taken by left $+$
- $+, -, *, /$ are left associative
- ^= are right associative
- String a+5*2 has two possible interpretations because of two different parse trees corresponding to $(a + 5) * 2$ and $a + (5 * 2)$.
- Precedence determines the correct interpretation.

Ambiguity

- Dangling else problem

Ambiguity

- Dangling else problem

$$Stmt \rightarrow if \quad expr \quad then \quad stmt$$
$$\mid if \quad expr \quad then \quad stmt \quad else \quad stmt$$

Ambiguity

- Dangling else problem
  $Stmt \rightarrow$ *if expr then stmt*
  $\mid$ *if expr then stmt else stmt*
- `if el then if e2 then S1 else S2` has two parse trees

Ambiguity

- Dangling else problem
  $Stmt \rightarrow$ *if expr then stmt*
  $\mid$ *if expr then stmt else stmt*
- `if el then if e2 then S1 else S2` has two parse trees

```
if(e1)
    if(e2)
        S1
    else
        S2
```

Ambiguity

- Dangling else problem
  $Stmt \rightarrow$ *if expr then stmt*
  $\mid$ *if expr then stmt else stmt*
- `if el then if e2 then S1 else S2` has two parse trees

```
if(e1)
    if(e2)
        S1
    else
        S2
```

```
if(e1)
    if(e2)
        S1
else
    S2
```

Resolving dangling `else` problem

- Match each `else` with the closest previous `then`

Resolving dangling `else` problem

- Match each `else` with the closest previous `then`
- *stmt* $\rightarrow$ `matched-stmt`
  | `unmatched-stmt`

Resolving dangling `else` problem

- Match each `else` with the closest previous `then`
- *stmt* $\rightarrow$ `matched-stmt`
  $\mid$ `unmatched-stmt`
- `matched-stmt` $\rightarrow$ `if expr then matched-stmt else matched-stmt`
  $\mid$*others*

Resolving dangling `else` problem

- Match each `else` with the closest previous `then`
- *stmt* → `matched-stmt`
  | `unmatched-stmt`
- `matched-stmt` → `if expr then matched-stmt else matched-stmt`
  | *others*
- `unmatched-stmt` → `if expr then stmt`
  | `if expr then matched-stmt else unmatched-stmt`

Parsing

- Process of determination whether a string can be generated by a grammar.

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:
  - **Top-down parsing:** Construction of the parse tree starts at the root (from the start symbol) and proceeds towards leaves (token or terminals). Ex ANTLR

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:
  - **Top-down parsing:** Construction of the parse tree starts at the root (from the start symbol) and proceeds towards leaves (token or terminals). Ex ANTLR (ANother Tool for Language Recognition)

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:
  - **Top-down parsing:** Construction of the parse tree starts at the root (from the start symbol) and proceeds towards leaves (token or terminals). Ex ANTLR (ANother Tool for Language Recognition)
  - **Bottom-up parsing:** Construction of the parse tree starts from the leaf nodes (tokens or terminals of the grammar) and proceeds towards root (start symbol). Ex YACC and BISON

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps
  - at a node labeled with non terminal A select one of the productions of A and construct children nodes (Which production?)

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps
  - at a node labeled with non terminal A select one of the productions of A and construct children nodes (Which production?)
  - find the next node at which subtree is Constructed (Which node?)

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps
  - at a node labeled with non terminal A select one of the productions of A and construct children nodes (Which production?)
  - find the next node at which subtree is Constructed (Which node?)